

Как приложение говорит с сервером

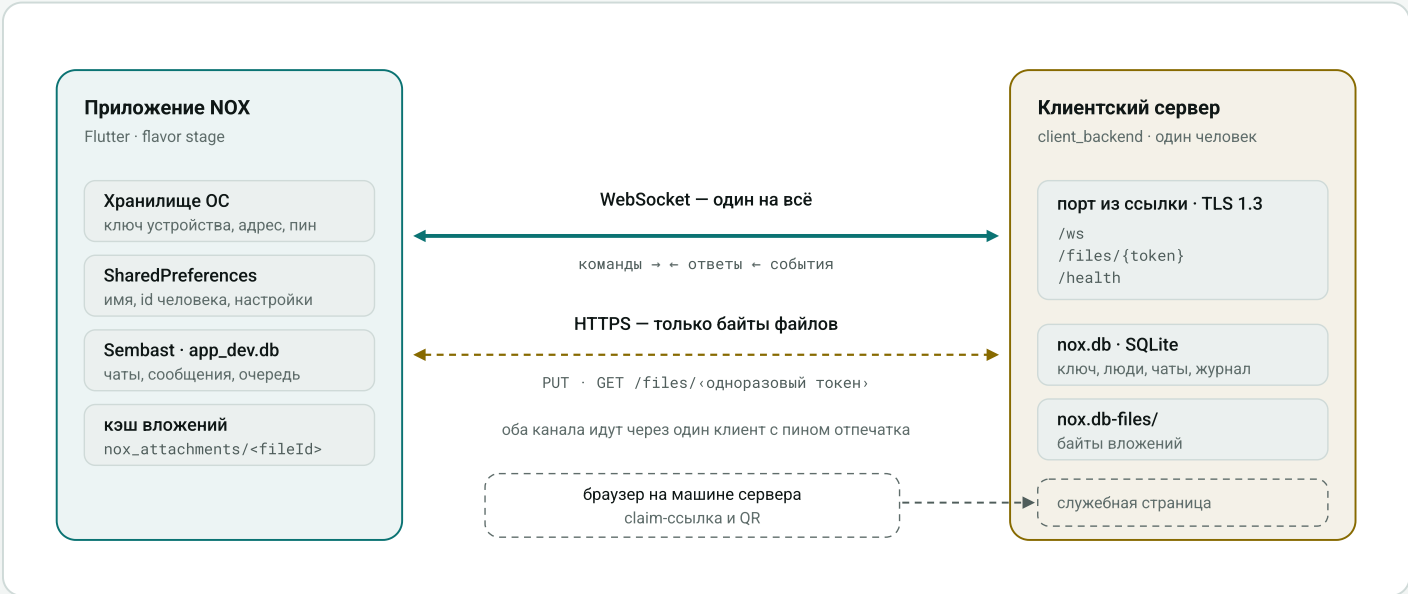
Всё идёт по **одному WebSocket**: команды, ответы и события. **HTTPS** нужен только для байтов файлов. Ниже — весь интерфейс, какие данные куда уходят и где они лежат с обеих сторон.

■ приложение, устройство ■ сервер ■ сбой · открытое хранение □ вне контракта

- Архитектура
- Доступность
- Формат обмена
- Соединение
- Команды
- События
- REST
- Файлы
- Синхронизация
- Ошибки
- Лимиты
- Какие данные куда
- База сервера
- Хранилища приложения

Архитектура

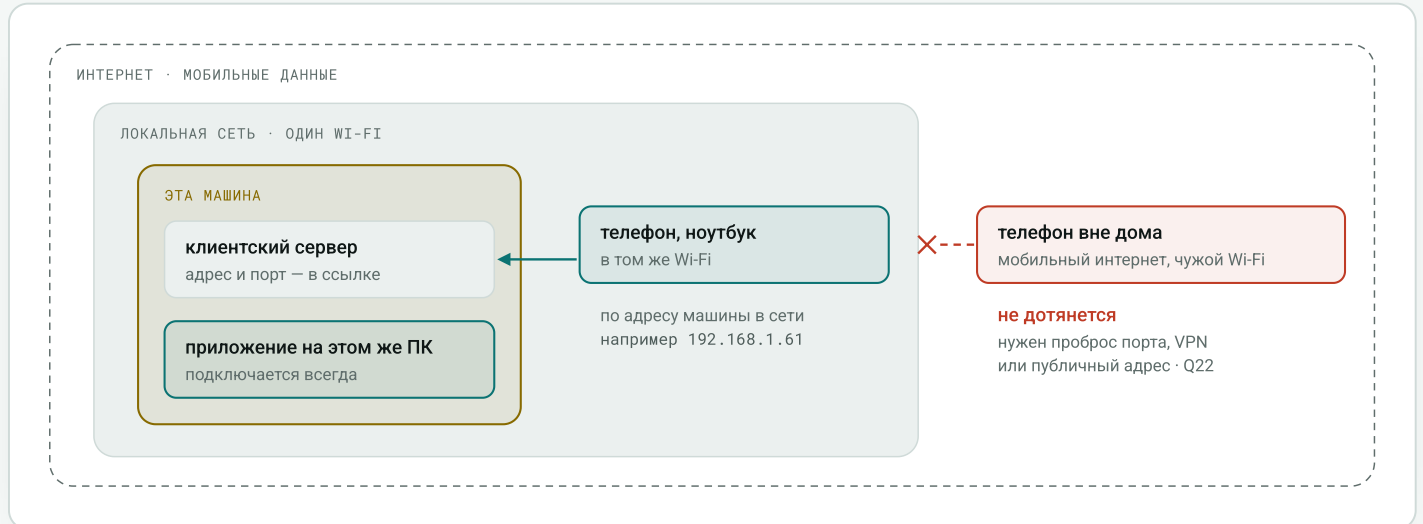
Два канала к одной машине, и у каждой стороны свои хранилища. **Клиентский сервер** — это `client_backend`: одна машина одного человека. Дальше — просто «сервер».



В приложении нет зашифрованного адреса сервера: адрес и порт оно берёт из ссылки спаривания.

Кто до сервера дотянется

Сервер работает в вашей **локальной сети**, и подключаются к нему устройства из той же сети: первое — по claim-ссылке, которую показывает сам сервер, остальные — по приглашению с уже спаренного устройства. По умолчанию сервер слушает только свою машину, и дотянется до него лишь приложение на ней же; как открыть его для сети — в `client_backend/README.md`.



Если сервер слушает сеть, QR на служебной странице сам подставляет в ссылку адрес машины в локальной сети. Если только свою машину — QR нет, ссылку вставляют в приложение на ней же. Выход в интернет не встроен — это открытый вопрос Q22.

АДРЕС ЗАШИТ В ССЫЛКУ

Если роутер выдаст серверу другой IP, устройства перестанут его находить: обновить адрес на устройстве нечем, поэтому нужно вернуть серверу прежний адрес. Значит, у машины сервера в сети должен быть постоянный адрес.

Формат обмена по WebSocket

Приложение и сервер обмениваются JSON-сообщениями по одному WebSocket-соединению. Каждое сообщение — одного из четырёх типов, тип определяется ключом верхнего уровня: `srv`, `cmd`, `ok` или `event`.

Вызов

`srv`

сервер → приложение

```
{
  "srv": {
    "schema_max": 1,
    "challenge": "q3ZkT8aR0vLx...9wN4Q=="
  }
}
```

<code>srv</code>	признак вызова; приходит один раз, сразу после подключения
<code>schema_max</code>	последняя версия протокола, которую знает сервер
<code>challenge</code>	32 случайных байта; устройство подписывает их своим ключом

Команда

`cmd`

приложение → сервер

```
{
  "id": 7,
  "cmd": "chat.create",
  "data": {
    "name": "Поездка"
  }
}
```

<code>id</code>	номер команды; с ним же придёт ответ
<code>cmd</code>	что сделать — имя команды
<code>data</code>	параметры команды; у каждой свои

Ответ

ok

сервер → приложение

УСПЕХ

```
{
  "id": 7,
  "ok": true,
  "data": {
    "chat": {
      "chat_id": "c_3f9a1c2e7b4d8a01",
      "name": "Поездка",
      "created_at": 1790491200,
      "created_by_label": "Anna",
      "last_message_preview": "",
      "last_activity_at": 1790491200
    }
  }
}
```

ОШИБКА

```
{
  "id": 7,
  "ok": false,
  "error": {
    "code": "name_taken",
    "message": "Chat name already exists"
  }
}
```

id	тот же номер, что у команды
ok	true — выполнено, false — ошибка
data	результат; только при успехе
error.code	код ошибки; по нему решает приложение
error.message	пояснение для человека

Событие

event

сервер → приложение

```
{
  "seq": 42,
  "event": "message.new",
  "data": {
    "message_id": "m_8b21d4e09c7f3a56",
    "seq": 42,
    "chat_id": "c_3f9a1c2e7b4d8a01",
    "author_id": "u_1d6e2b9f40c8a7e3",
    "author_label": "Anna",
    "client_message_id": "3b8f1d2e-9c4a-4f6b-a7e1-5d2c8b0f4a93",
    "sent_at": 1790491260,
    "body": {
      "type": "text",
      "text": "Привет"
    }
  }
}
```

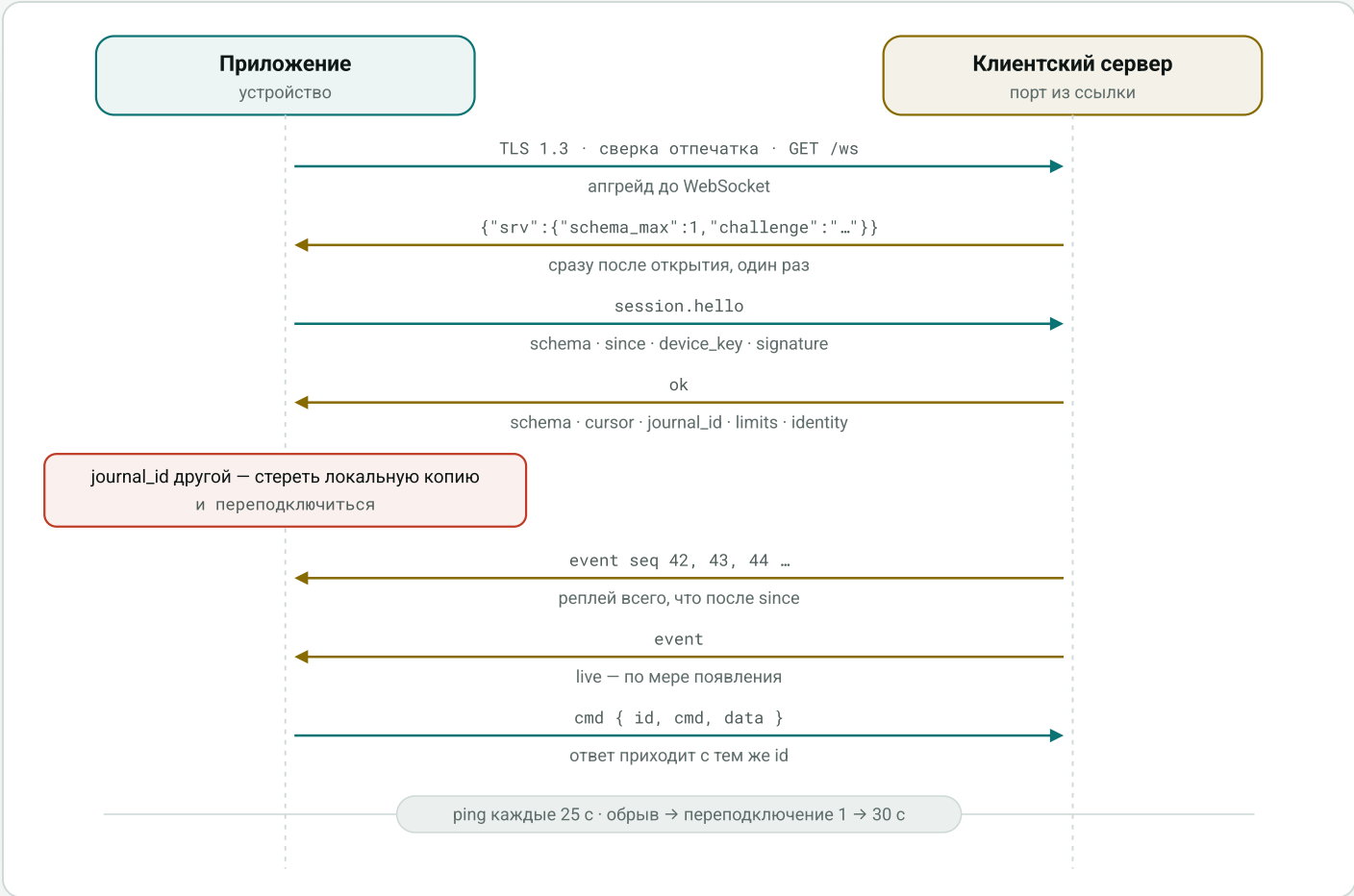
seq номер в журнале сервера; приложение хранит его как курсор. 0 — событие только вживую

event что произошло — тип события

data содержимое: сообщение, чат и т. п.

Жизнь соединения

До `session.hello` сервер принимает только одну команду — `pair`. На остальные отвечает `invalid_request`, поэтому приложение держит их, пока не придёт ответ на hello, но не дольше 10 секунд.



Как устроены пин и подпись — во втором документе, [NOX Pinned Channel](#) (pinned-channel.pdf).

Команды

16 команд, все по WebSocket. Сверху — сводка, ниже — каждая команда с примером запроса и ответа. Стрелкой отмечено событие, которое команда порождает; `?` — необязательное поле.

КОМАНДА	ЗАЧЕМ
СЕССИЯ	
<code>session.hello</code>	вход по подписи; реплей с курсора, дальше live
<code>pair</code> (до hello)	потратить токен и зарегистрировать ключ устройства → <code>device.paired</code> другим устройствам
ЛИЧНОСТЬ	
<code>identity.setLabel</code>	переименовать человека → <code>identity.updated</code> другим устройствам
УСТРОЙСТВА	
<code>device.list</code>	устройства этого человека

КОМАНДА	ЗАЧЕМ
<code>device.invite</code>	приглашение ещё одного устройства на 10 минут
<code>device.revoke</code>	удалить ключ, погасить приглашения, оборвать его соединения → <code>device.revoked</code> отозванному
ЧАТЫ	
<code>chats.list</code>	список по последней активности, ≤ 100 на страницу
<code>chat.get</code>	один чат
<code>chat.create</code>	новый чат; имя уникально без учёта регистра → <code>chat.created</code>
<code>chat.rename</code>	переименовать → <code>chat.updated</code> , если имя изменилось
<code>chat.nameAvailable</code>	проверка имени до создания
СООБЩЕНИЯ	
<code>messages.list</code>	история назад по seq
<code>message.send</code>	отправить; повтор с тем же <code>client_message_id</code> вернёт оригинал → <code>message.new</code>
ФАЙЛЫ	
<code>file.uploadBegin</code>	пропуск на PUT
<code>file.downloadBegin</code>	пропуск на GET
<code>chat.files</code>	вложения чата; есть на сервере, приложение не вызывает

ОБЩИЕ ТИПЫ

Chat

карточка чата — в ответах `chats.list`, `chat.get`, `chat.create`, `chat.rename` и в событиях `chat.created`, `chat.updated`

<code>chat_id</code>	<code>string</code> · <code>c_...</code>	идентификатор чата
<code>name</code>	<code>string</code>	имя, 1–64 символа
<code>created_at</code>	<code>int</code> · <code>unix-сек</code>	когда создан
<code>created_by_label</code>	<code>string</code>	имя создателя на момент создания
<code>last_message_preview</code>	<code>string</code>	текст или имя файла последнего сообщения, до 120 символов
<code>last_activity_at</code>	<code>int</code> · <code>unix-сек</code>	последнее сообщение или создание чата

Message

сообщение — в ответах `messages.list`, `message.send` и в событии `message.new`

<code>message_id</code>	<code>string</code> · <code>m_...</code>	идентификатор сообщения
<code>seq</code>	<code>int</code>	номер его события в журнале сервера
<code>chat_id</code>	<code>string</code> · <code>c_...</code>	в каком чате
<code>author_id</code>	<code>string</code> · <code>u_...</code>	кто написал
<code>author_label</code>	<code>string</code>	имя автора на момент отправки
<code>client_message_id?</code>	<code>string</code>	ключ отправки; только в своих сообщениях
<code>sent_at</code>	<code>int</code> · <code>unix-сек</code>	когда сервер принял
<code>body?</code>	<code>object</code>	тело как прислал отправитель: <code>{"type": "text", "text": ...}</code>
<code>attachment?</code>	<code>object</code>	вложение: <code>file_id</code> , <code>name</code> , <code>size</code> , <code>mime</code> , <code>expires_at</code>

СЕССИЯ

session.hello → после ответа — реплей событий новее <code>since</code>, затем <code>live</code> Приветствие: подписью доказать ключ устройства, получить курсор, пределы и личность
--

ЗАПРОС

```
{
  "id": 1,
  "cmd": "session.hello",
  "data": {
    "schema": 1,
    "since": 40,
    "device_key":
"A6EHv/POEL4dcN0Y50vAmWfk1jCbPQ1fHdyGZBJVMbg=",
    "signature":
"puHNSjLy6sqx5MEgiAuZiz6Zuyjohrz0EIEjkdQ9iazgyIEeUvGbK+UA
IhHjpscFPCMBN7rpbfeoXr1UZXQZBQ=="
  }
}
```

ПАРАМЕТРЫ · DATA

schema	int	версия контракта; иное → unsupported_schema
since?	int	последний применённый seq; без поля — replay нет
label?	string	устаревшее: переименовать при входе; приложение не шлёт
device_key	string · base64	публичный ключ этого устройства
signature	string · base64	подпись "nox/challenge/v1:" сырые байты challenge

ОТВЕТ

```
{
  "id": 1,
  "ok": true,
  "data": {
    "schema": 1,
    "cursor": 42,
    "journal_id": "j_7a1e4c90b3d25f68",
    "limits": {
      "max_message_bytes": 65536,
      "max_attachment_bytes": 104857600,
      "max_frame_bytes": 131072
    },
    "identity": {
      "id": "u_1d6e2b9f40c8a7e3",
      "label": "Anna"
    }
  }
}
```

ОТВЕТ · DATA

schema	int	версия контракта сервера
cursor	int	последний seq журнала на момент ответа
journal_id	string · j_...	идентификатор хранилища; смена → сбросить кэш
limits	object	пределы, объявленные сервером
limits.max_message_bytes	int · байты	максимум для body сообщения целиком: байты JSON, как их прислали
limits.max_attachment_bytes	int · байты	максимальный размер вложения
limits.max_frame_bytes	int · байты	максимум для WebSocket-сообщения целиком
identity	object	кто это подключение (без created)
identity.id	string · u_...	собственный author-id человека
identity.label	string	текущее имя человека

ОШИБКИ invalid_request unsupported_schema unauthenticated internal

pair до hello → device.paired другим устройствам человека

Спаривание: потратить токен из ссылки и привязать ключ устройства к человеку

ЗАПРОС

```
{
  "id": 1,
  "cmd": "pair",
  "data": {
    "token": "ymDpgoGo4EVu1RXjvRW6Iw",
    "device_key":
"A6EHv/POEL4dcN0Y50vAmWfk1jCbPQ1fHdyGZBJVMbg=",
    "platform": "ios"
  }
}
```

ОТВЕТ

```
{
  "id": 1,
  "ok": true,
  "data": {
    "identity": {
      "id": "u_1d6e2b9f40c8a7e3",
      "label": "User7421",
      "created": true
    }
  }
}
```

ПАРАМЕТРЫ · DATA

token	string · base64url	одноразовый claim-токен или приглашение устройства
device_key	string · base64	публичный ключ привязываемого устройства
platform	string	семейство ОС, без модели железа

ОШИБКИ

invalid_request

invalid_token

token_expired

internal

ОТВЕТ · DATA

identity	object	личность, к которой привязано устройство
identity.id	string · u_...	author-id человека
identity.label	string	имя; новому человеку — User<4 цифры>
identity.created	bool	эта операция завела человека → экран имени

ЛИЧНОСТЬ

identity.setLabel → identity.updated другим устройствам человека

Переименовать человека; уникальность имени не проверяется

ЗАПРОС

```
{
  "id": 5,
  "cmd": "identity.setLabel",
  "data": {
    "label": "Anna"
  }
}
```

ПАРАМЕТРЫ · DATA

label string новое имя человека

ОШИБКИ

invalid_request

internal

ОТВЕТ

```
{
  "id": 5,
  "ok": true,
  "data": {
    "label": "Anna"
  }
}
```

ОТВЕТ · DATA

label string записанное имя (после trim)

УСТРОЙСТВА

device.list

Список устройств этого человека, старейшее первым; data не читается

ЗАПРОС

```
{
  "id": 7,
  "cmd": "device.list",
  "data": {}
}
```

ОТВЕТ

```
{
  "id": 7,
  "ok": true,
  "data": {
    "devices": [
      {
        "device_key":
"A6EHv/POEL4dcN0Y50vAmWfk1jCbPQ1fHdyGZBJVMbg=",
        "platform": "ios",
        "created_at": 1790491200,
        "last_seen_at": 1790491260
      }
    ]
  }
}
```

ПАРАМЕТРЫ · DATA

нет полей

ОШИБКИ

invalid_request

internal

ОТВЕТ · DATA

devices	array	все устройства этого человека
devices[].device_key	string · base64	ключ устройства; своё клиент узнаёт сам
devices[].platform	string	семейство ОС из последнего pair
devices[].created_at	int · unix-сек	момент первой привязки устройства
devices[].last_seen_at	int · unix-сек	последнее приветствие или pair

device.invite

Выпустить приглашение для нового устройства вместе с готовой ссылкой; data не читается

ЗАПРОС

```
{
  "id": 8,
  "cmd": "device.invite",
  "data": {}
}
```

ОТВЕТ

```
{
  "id": 8,
  "ok": true,
  "data": {
    "token": "H5DzkaMPP0_ZmTJroQhneA",
    "link": "https://nox.app/p/#AQHAqAE9H5DzkaMPP0_ZmTJroQhneNuS9XC2M
Za_5Du-4GU4er-MHh-Q85GjDz9P2Zkya6EIZ3g"
  }
}
```

ПАРАМЕТРЫ · DATA

нет полей

ОШИБКИ

invalid_request

internal

ОТВЕТ · DATA

token	string · base64url	одноразовый токен на 10 минут; уже внутри link
link	string · URL	ссылка для QR на новое устройство

device.revoke → device.revoked отозванному устройству; его соединение закрывается

Отозвать ключ устройства и сразу оборвать его живое соединение

ЗАПРОС

```
{
  "id": 9,
  "cmd": "device.revoke",
  "data": {
    "device_key": "Zq9mTf3K8wYb2pR5nL1vX7cJ4hD6sA0uE9i03gW8yBk="
  }
}
```

ОТВЕТ

```
{
  "id": 9,
  "ok": true,
  "data": {}
}
```

ПАРАМЕТРЫ · DATA

device_key string · base64 ключ отзываемого устройства

ОШИБКИ

invalid_request

not_found

internal

ОТВЕТ · DATA

нет полей

ЧАТЫ

chats.list

Страница списка чатов с необязательным поиском по имени

ЗАПРОС

```
{
  "id": 2,
  "cmd": "chats.list",
  "data": {
    "page": 1,
    "page_size": 20,
    "query": "поезд"
  }
}
```

ПАРАМЕТРЫ · DATA

page int номер страницы; меньше 1 — invalid_request

page_size int сколько чатов на странице

query? string подстрока имени, без учёта регистра (Unicode)

ОШИБКИ invalid_request internal

ОТВЕТ

```
{
  "id": 2,
  "ok": true,
  "data": {
    "chats": [
      {
        "chat_id": "c_3f9a1c2e7b4d8a01",
        "name": "Поездка",
        "created_at": 1790491200,
        "created_by_label": "Anna",
        "last_message_preview": "Привет",
        "last_activity_at": 1790491260
      }
    ],
    "has_more": false
  }
}
```

ОТВЕТ · DATA

chats [Chat\[\]](#) порядок: last_activity_at ↓, затем chat_id ↑

has_more bool есть ли следующая страница

chat.get

Карточка одного чата по id (например, чата вне загруженных страниц)

ЗАПРОС

```
{
  "id": 3,
  "cmd": "chat.get",
  "data": {
    "chat_id": "c_9e4b7d21a0f6c35e"
  }
}
```

ПАРАМЕТРЫ · DATA

chat_id string · c_... какой чат вернуть

ОШИБКИ invalid_request not_found internal

ОТВЕТ

```
{
  "id": 3,
  "ok": true,
  "data": {
    "chat": {
      "chat_id": "c_9e4b7d21a0f6c35e",
      "name": "Работа",
      "created_at": 1790487600,
      "created_by_label": "Anna",
      "last_message_preview": "",
      "last_activity_at": 1790487600
    }
  }
}
```

ОТВЕТ · DATA

chat [Chat](#) текущая карточка чата

chat.create → chat.created всем устройствам, включая отправителя

Создать чат; сервер решает, свободно ли имя

ЗАПРОС

```
{
  "id": 4,
  "cmd": "chat.create",
  "data": {
    "name": "Поездка"
  }
}
```

ПАРАМЕТРЫ · DATA

name string имя нового чата, уникальное без учёта регистра

ОШИБКИ

invalid_request

name_taken

internal

ОТВЕТ

```
{
  "id": 4,
  "ok": true,
  "data": {
    "chat": {
      "chat_id": "c_3f9a1c2e7b4d8a01",
      "name": "Поездка",
      "created_at": 1790491200,
      "created_by_label": "Anna",
      "last_message_preview": "",
      "last_activity_at": 1790491200
    }
  }
}
```

ОТВЕТ · DATA

chat [Chat](#) созданный чат

chat.rename → chat.updated всем, если имя изменилось

Переименовать чат; уникальность проверяется без самого чата

ЗАПРОС

```
{
  "id": 5,
  "cmd": "chat.rename",
  "data": {
    "chat_id": "c_3f9a1c2e7b4d8a01",
    "name": "Поездка в Карпаты"
  }
}
```

ПАРАМЕТРЫ · DATA

chat_id string · c_... какой чат переименовать

name string новое имя, уникальное без учёта регистра

ОШИБКИ

invalid_request

not_found

name_taken

internal

ОТВЕТ

```
{
  "id": 5,
  "ok": true,
  "data": {
    "chat": {
      "chat_id": "c_3f9a1c2e7b4d8a01",
      "name": "Поездка в Карпаты",
      "created_at": 1790491200,
      "created_by_label": "Anna",
      "last_message_preview": "Привет",
      "last_activity_at": 1790491260
    }
  }
}
```

ОТВЕТ · DATA

chat [Chat](#) карточка чата после переименования

chat.nameAvailable

Предварительная проверка имени чата; окончательно решают create/rename

ЗАПРОС

```
{
  "id": 6,
  "cmd": "chat.nameAvailable",
  "data": {
    "name": "работа",
    "exclude_chat_id": "c_3f9a1c2e7b4d8a01"
  }
}
```

ПАРАМЕТРЫ · DATA

name string проверяемое имя

exclude_chat_id? string · c_... не учитывать этот чат (переименование)

ОТВЕТ

```
{
  "id": 6,
  "ok": true,
  "data": {
    "available": false
  }
}
```

ОТВЕТ · DATA

available bool свободно ли имя без учёта регистра

СООБЩЕНИЯ

messages.list

Порция истории чата, от конца назад по seq

ЗАПРОС

```
{
  "id": 11,
  "cmd": "messages.list",
  "data": {
    "chat_id": "c_3f9a1c2e7b4d8a01",
    "limit": 20
  }
}
```

ОТВЕТ

```
{
  "id": 11,
  "ok": true,
  "data": {
    "messages": [
      {
        "message_id": "m_8b21d4e09c7f3a56",
        "seq": 42,
        "chat_id": "c_3f9a1c2e7b4d8a01",
        "author_id": "u_1d6e2b9f40c8a7e3",
        "author_label": "Anna",
        "client_message_id": "3b8f1d2e-9c4a-4f6b-a7e1-5d2c8b0f4a93",
        "sent_at": 1790491260,
        "body": {
          "type": "text",
          "text": "Привет"
        },
        "attachment": {
          "file_id": "f_5c0e7a91d2b4f836",
          "name": "photo.jpg",
          "size": 184320,
          "mime": "image/jpeg",
          "expires_at": 2105851200
        }
      }
    ],
    "has_more": true
  }
}
```

ПАРАМЕТРЫ · DATA

chat_id	string · c_...	чат, чью историю читаем
before_seq?	int	только сообщения с seq меньше этого
limit	int	размер порции

ОТВЕТ · DATA

messages	Message[]	порция, по возрастанию seq
has_more	bool	есть ли сообщения старше порции

message.send → message.new всем устройствам, включая отправителя

Отправить текст и/или вложение; повтор с тем же ключом возвращает прежнее эхо

ЗАПРОС

```
{
  "id": 10,
  "cmd": "message.send",
  "data": {
    "chat_id": "c_3f9a1c2e7b4d8a01",
    "client_message_id": "3b8f1d2e-9c4a-4f6b-a7e1-5d2c8b0f4a93",
    "body": {
      "type": "text",
      "text": "Привет"
    },
    "attachment": {
      "file_id": "f_5c0e7a91d2b4f836"
    }
  }
}
```

ПАРАМЕТРЫ · DATA

chat_id	string · c_...	чат, куда пишем
client_message_id	string	ключ идемпотентности, уникален на человека
body?	object	тело; нужно body или attachment
body.type?	string	сейчас "text"; сервер не проверяет
body.text?	string	текст сообщения; сервер не проверяет
attachment?	object	вложение; нужно body или attachment
attachment.file_id	string · f_...	загруженный, ещё не привязанный файл

ОШИБКИ

invalid_request

not_found

payload_too_large

internal

ОТВЕТ

```
{
  "id": 10,
  "ok": true,
  "data": {
    "message": {
      "message_id": "m_8b21d4e09c7f3a56",
      "seq": 42,
      "chat_id": "c_3f9a1c2e7b4d8a01",
      "author_id": "u_1d6e2b9f40c8a7e3",
      "author_label": "Anna",
      "client_message_id": "3b8f1d2e-9c4a-4f6b-a7e1-5d2c8b0f4a93",
      "sent_at": 1790491260,
      "body": {
        "type": "text",
        "text": "Привет"
      },
      "attachment": {
        "file_id": "f_5c0e7a91d2b4f836",
        "name": "photo.jpg",
        "size": 184320,
        "mime": "image/jpeg",
        "expires_at": 2105851200
      }
    }
  }
}
```

ОТВЕТ · DATA

message [Message](#) эхо сохранённого; поля как в messages.list

ФАЙЛЫ

file.uploadBegin

Заявить файл и получить одноразовый адрес для PUT ровно size байт

ЗАПРОС

```
{
  "id": 9,
  "cmd": "file.uploadBegin",
  "data": {
    "name": "photo.jpg",
    "size": 184320,
    "mime": "image/jpeg"
  }
}
```

ОТВЕТ

```
{
  "id": 9,
  "ok": true,
  "data": {
    "file_id": "f_5c0e7a91d2b4f836",
    "upload_url":
"/files/kP4xR8mV2nQ7wL0sT5yB9cF3hJ6dG1zA8eU4i07pN2r",
    "upload_token":
"kP4xR8mV2nQ7wL0sT5yB9cF3hJ6dG1zA8eU4i07pN2r",
    "max_attachment_bytes": 104857600
  }
}
```

ПАРАМЕТРЫ · DATA

name	string	имя файла, уйдёт в attachment
size	int	точный размер в байтах
mime	string	MIME-тип; клиент выводит из расширения

ОШИБКИ

invalid_request

payload_too_large

internal

ОТВЕТ · DATA

file_id	string · f_...	id файла для message.send
upload_url	string · путь	относительный путь для PUT байтов
upload_token	string · base64url	одноразовый, 10 минут, только PUT
max_attachment_bytes	int · байты	потолок размера файла, сейчас 100 МиБ

file.downloadBegin

Получить одноразовый адрес для GET байтов файла (с Range)

ЗАПРОС

```
{
  "id": 13,
  "cmd": "file.downloadBegin",
  "data": {
    "file_id": "f_5c0e7a91d2b4f836"
  }
}
```

ПАРАМЕТРЫ · DATA

file_id	string · f_...	файл с загруженными байтами
---------	----------------	-----------------------------

ОШИБКИ

invalid_request

not_found

attachment_gone

internal

ОТВЕТ

```
{
  "id": 13,
  "ok": true,
  "data": {
    "download_url":
"/files/Tz3mQ9vW1bK6nX4pR8cL2hY7dF0sJ5gA9eV3uB6iM2w",
    "download_token":
"Tz3mQ9vW1bK6nX4pR8cL2hY7dF0sJ5gA9eV3uB6iM2w"
  }
}
```

ОТВЕТ · DATA

download_url	string · путь	относительный путь для GET байтов
download_token	string · base64url	одноразовый, 10 минут, только GET

chat.files

Вложения чата страницами; есть на сервере, приложение пока не вызывает

ЗАПРОС

```
{
  "id": 12,
  "cmd": "chat.files",
  "data": {
    "chat_id": "c_3f9a1c2e7b4d8a01",
    "limit": 20
  }
}
```

ОТВЕТ

```
{
  "id": 12,
  "ok": true,
  "data": {
    "files": [
      {
        "file_id": "f_5c0e7a91d2b4f836",
        "name": "photo.jpg",
        "size": 184320,
        "mime": "image/jpeg",
        "expires_at": 2105851200,
        "message_id": "m_8b21d4e09c7f3a56",
        "seq": 42
      }
    ],
    "has_more": false
  }
}
```

ПАРАМЕТРЫ · DATA

chat_id	string · c_...	чат, чьи вложения читаем
before_seq?	int	только вложения с seq меньше этого
limit	int	размер порции

ОТВЕТ · DATA

files	array	вложения, по возрастанию seq
files[].file_id	string · f_...	id файла для file.downloadBegin
files[].name	string	имя файла из uploadBegin
files[].size	int · байты	размер из uploadBegin
files[].mime	string	тип из uploadBegin
files[].expires_at	int · unix-сек	срок хранения байтов (сейчас +10 лет)
files[].message_id	string · m_...	сообщение, к которому привязан файл
files[].seq	int	seq этого сообщения; курсор для before_seq
has_more	bool	есть ли вложения старше порции

ОШИБКИ

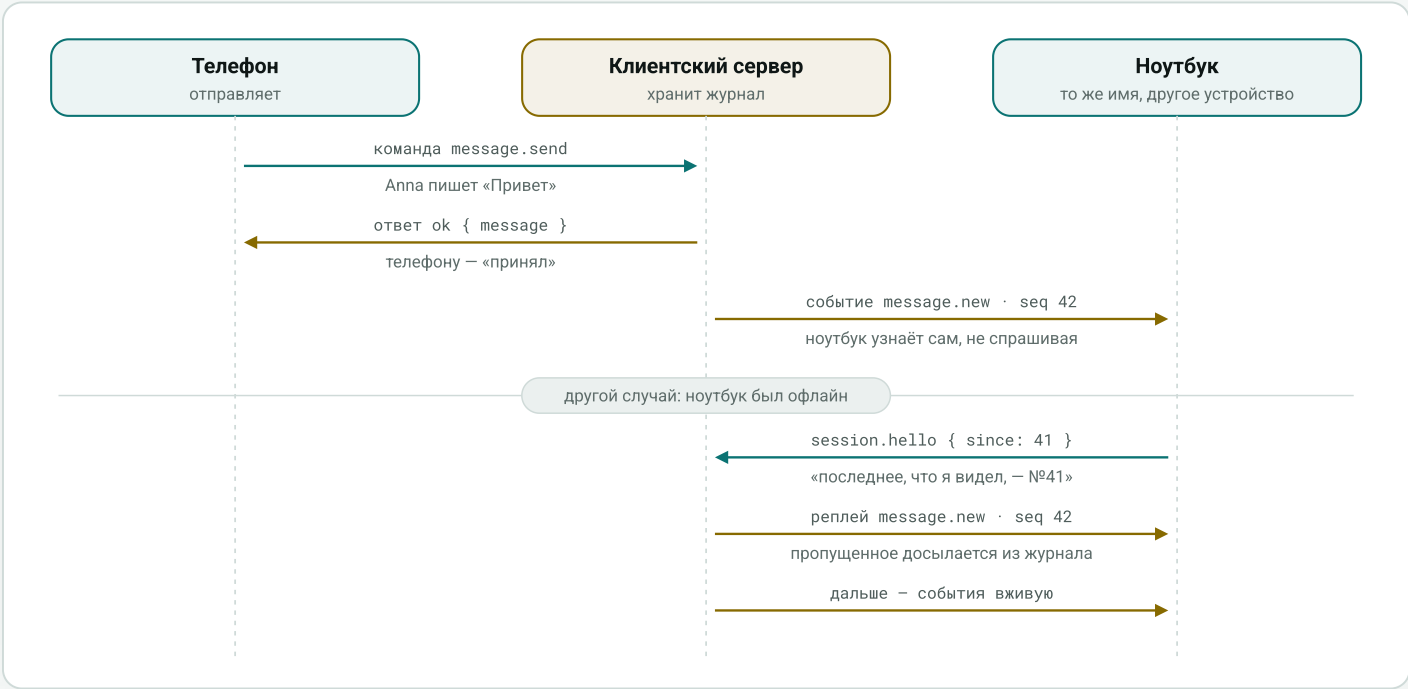
invalid_request

not_found

internal

События

Событие — это сообщение, которое сервер присылает **сам, без запроса**.
Команда — вопрос и ответ, событие — новость: так устройства человека узнают о том, что произошло на других.



Ответ получает только тот, кто спросил. Событие получают все, кому оно нужно, — в том числе устройства, которые в этот момент были офлайн, если событие журнальное.

Журнальные seq > 0

`chat.created` · `chat.updated` · `message.new`

Хранятся на сервере под номером. Устройство, которое было офлайн, при подключении получает всё пропущенное — **не теряются никогда**.

Живые seq 0

`device.revoked` · `identity.updated` · `device.paired`

Приходят только тем, кто сейчас на связи, и не хранятся. Пропустить не страшно: нужное устройство узнает иначе — список устройств перечитывается при открытии экрана, имя приходит в ответе на приветствие.

Правило: всё, что касается переписки, — журнальное. Всё служебное — живое.

chat.created журнал · seq > 0

Появился новый чат — приложение добавляет его в список. кому: всем устройствам, включая то, что создало чат

```
{
  "seq": 43,
  "event": "chat.created",
  "data": {
    "chat_id": "c_9e4b7d21a0f6c35e",
    "name": "Работа",
    "created_at": 1790491320,
    "created_by_label": "Anna",
    "last_message_preview": "",
    "last_activity_at": 1790491320
  }
}
```

seq int номер в журнале; приложение запоминает как курсор
data [Chat](#) новый чат

chat.updated журнал · seq > 0

Чат переименовали. На новые сообщения это событие не приходит. кому: всем устройствам, если имя действительно изменилось

```
{
  "seq": 44,
  "event": "chat.updated",
  "data": {
    "chat_id": "c_9e4b7d21a0f6c35e",
    "name": "Работа 2026",
    "created_at": 1790491320,
    "created_by_label": "Anna",
    "last_message_preview": "",
    "last_activity_at": 1790491320
  }
}
```

seq int номер в журнале
data [Chat](#) чат целиком, уже с новым именем

message.new журнал · seq > 0

Сервер сохранил сообщение — приложение показывает его в чате. кому: всем устройствам, включая отправившее

```
{
  "seq": 42,
  "event": "message.new",
  "data": {
    "message_id": "m_8b21d4e09c7f3a56",
    "seq": 42,
    "chat_id": "c_3f9a1c2e7b4d8a01",
    "author_id": "u_1d6e2b9f40c8a7e3",
    "author_label": "Anna",
    "client_message_id": "3b8f1d2e-9c4a-4f6b-a7e1-5d2c8b0f4a93",
    "sent_at": 1790491260,
    "body": {
      "type": "text",
      "text": "Привет"
    }
  }
}
```

seq int номер в журнале
data [Message](#) сообщение целиком

device.revoked живую · seq 0

Это устройство отозвали — приложение выходит из аккаунта и стирает локальные данные. кому: отозванному устройству, прямо перед закрытием соединения

```
{
  "seq": 0,
  "event": "device.revoked",
  "data": {
    "device_key":
      "Zq9mTf3K8wYb2pR5nL1vX7cJ4hD6sA0uE9i03gW8yBk="
  }
}
```

seq int всегда 0 — событие не хранится
data.device_key string · base64 какой ключ отозван

identity.updated

вживую · seq 0

Человек сменил имя на другом устройстве — приложение обновляет его у себя.

кому: другим устройствам человека

```
{
  "seq": 0,
  "event": "identity.updated",
  "data": {
    "label": "Anna"
  }
}
```

seq int всегда 0
data.label string новое имя

device.paired

вживую · seq 0

Добавилось устройство — открытый экран устройств перечитывает список.

кому: другим устройствам человека

```
{
  "seq": 0,
  "event": "device.paired",
  "data": {}
}
```

seq int всегда 0
data object пустой: это только сигнал перечитать device.list

REST

HTTP нужен для четырёх вещей: открыть WebSocket, переслать байты файла, ответить на проверку живости и показать человеку служебную страницу.

ЭНДПОИНТ	ЗАЧЕМ
<u>GET</u> /ws	апгрейд до WebSocket — единственный канал команд
<u>PUT</u> /files/{token}	загрузить байты: ровно заявленный размер → 204
<u>GET</u> /files/{token}	скачать байты, поддерживает Range
<u>GET</u> /health	{ <code>"status": "ok"</code> }
<u>GET</u> /	страница для человека у сервера: состояние, claim-ссылка и QR; не часть контракта

GET /ws **основной** · TLS 1.3

Открыть WebSocket — единственный канал команд.

кто пускает: на уровне HTTP никто: сервер подтверждает себя пином TLS, устройство — подписью в session.hello (или токеном в pair).

ЗАПРОС

```
GET /ws HTTP/1.1
Host: 192.168.1.61:8080
Connection: Upgrade
Upgrade: websocket
Sec-WebSocket-Version: 13
Sec-WebSocket-Key: dGhlIHhnbXBsZSBub25jZQ==
Sec-WebSocket-Extensions: permessage-deflate;
client_max_window_bits
```

ОТВЕТ

```
HTTP/1.1 101 Switching Protocols
Connection: Upgrade
Upgrade: websocket
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=

<тела нет — дальше WebSocket; первым сервер шлёт srv>
```

ПОЛЯ ЗАПРОСА

Host	обяз.	сверяется с Origin; попадает в ссылку device.invite, если сервер слушает все интерфейсы
Connection	обяз.	Upgrade — без него 426
Upgrade	обяз.	websocket — без него 426
Sec-WebSocket-Version	обяз.	только 13, иначе 400
Sec-WebSocket-Key	обяз.	16 случайных байт в base64, ровно один
Origin		если есть — его host:port обязан совпасть с Host, иначе 403; приложение не шлёт
Sec-WebSocket-Extensions		приложение предлагает deflate, сервер молча отказывает — сжатия нет
Sec-WebSocket-Protocol		подпротоколов нет; приложение не шлёт
тело		нет; строка запроса игнорируется — курсор since едет в session.hello, а не в URL

КОДЫ ОТВЕТА

101 апгрейд принят; следом вызов srv

426 нет Connection: Upgrade или Upgrade: websocket, либо HTTP/1.0

400 версия не 13 (в ответе Sec-WebSocket-Version: 13); ключ отсутствует, повторён или не 16 байт

403 Origin есть и не совпадает с Host, в том числе Origin: null

405 метод не GET

307 · 400 · 431 · 505 ответы самого HTTP-сервера, до NOX: неканонический путь (/ws → Location: /ws), битый запрос или нет Host, заголовки больше 1 МБ, не HTTP/1.x

ПОЛЯ ОТВЕТА

Sec-WebSocket-Accept	base64(SHA-1(ключ + GUID)) — клиент сверяет
Upgrade	websocket — апгрейд подтверждён; Extensions и Protocol в ответе нет
тело	нет; дальше сообщения WebSocket (см. «Формат обмена»)

ВАЖНО

- После 101 ошибки приходят только кодами закрытия: 1000 — устройство отозвано или связь оборвалась, 1001 — сервер останавливается, 1002 — сообщение не разбирается, 1008 — клиент не успевает читать, 1009 — сообщение больше 128 KiB, 1011 — внутренняя ошибка.
- Ping каждые 25 с с обеих сторон; pong сервер ждёт 5 с, иначе закрывает.
- До session.hello сервер принимает только pair; остальное получает invalid_request, соединение не рвётся.

PUT

/files/{token}

основной

· TLS 1.3

Залить байты файла, заявленного в file.uploadBegin.

кто пускает: одноразовый upload_token в пути; заголовков авторизации нет.

ЗАПРОС

```
PUT /files/kP4xR8mV2nQ7wL0sT5yB9cF3hJ6dG1zA8eU4i07pN2r
HTTP/1.1
Host: 192.168.1.61:8080
Content-Length: 184320

<184320 байт файла photo.jpg>
```

ПОЛЯ ЗАПРОСА

token	в пути	upload_token из file.uploadBegin (upload_url — готовый путь): 32 байта base64url, один раз, 10 минут
Host	обяз.	обязателен в HTTP/1.1; значение не сверяется
Content-Length		длина тела; с size не сверяется — решает фактическое число байт
Transfer-Encoding		chunked вместо длины — можно; другое значение — 501
Content-Type		игнорируется; тип уже заявлен в uploadBegin
тело		сырые байты файла, ровно size из uploadBegin; без multipart и обёрток

ОТВЕТ

```
HTTP/1.1 204 No Content
Date: Sun, 27 Sep 2026 10:15:02 GMT
```

ПОЛЯ ОТВЕТА

Content-Type	только у ошибок: text/plain, одна строка
Connection	close — после 413 и после обрыва
тело	204 — пусто; ошибка — одна строка текста

КОДЫ ОТВЕТА

- 204** ровно size байт на диске (fsync), файл помечен залитым — теперь file_id годится для message.send
 - 400** байт меньше size («byte count does not match the declared size») или обрыв и таймаут («upload interrupted»)
 - 413** байт больше size; соединение закрывается
 - 404** токен неизвестен, истёк, уже погашен, выдан на скачивание или сгорел при рестарте — причина не раскрывается
 - 500** сбой хранилища («storage failure»)
- 405 · 307 · 400 · 431 · 501** ответы самого HTTP-сервера, до NOX: чужой метод, неканонический путь, битый запрос, огромные заголовки, Transfer-Encoding не chunked

ВАЖНО

- Токен гасится первым обращением при любом исходе — даже PUT с токеном скачивания сжигает его. Повтор — только через новый file.uploadBegin, и у файла будет новый file_id.
- Токен должен быть жив в момент запроса (10 минут); на само тело даётся 15 минут.
- Content-Length и Content-Type с size и mime не сверяются: 204 получает только тело ровно заявленной длины.

GET /files/{token} **основной · TLS 1.3**

Скачать байты файла; поддерживает Range.
кто пускает: одноразовый download_token в пути; заголовков авторизации нет.

ЗАПРОС

```
GET /files/Tz3mQ9vW1bK6nX4pR8cL2hY7dF0sJ5gA9eV3uB6iM2w
HTTP/1.1
Host: 192.168.1.61:8080
Accept-Encoding: gzip
```

ПОЛЯ ЗАПРОСА

token	в пути	download_token из file.downloadBegin (download_url — готовый путь): один раз, 10 минут
Host	обяз.	обязателен в HTTP/1.1; значение не сверяется
Range		bytes=N- — докачка с байта N; приложение не шлёт
If-...		стандартные условные заголовки (If-Range, If-Modified-Since и др.); приложение не шлёт
Accept-Encoding		приложение шлёт gzip — сжатия нет
тело		нет

КОДЫ ОТВЕТА

- 200** весь файл
 - 206** Range: bytes=131072- → Content-Range: bytes 131072-184319/184320; несколько диапазонов — multipart/byteranges
 - 304 · 412** ответ на условные заголовки, без тела
 - 416** диапазон за концом файла → Content-Range: bytes */184320
 - 404** токен неизвестен, истёк, погашен, выдан на загрузку или сгорел при рестарте; файл не залит или байтов нет на диске
 - 405** HEAD — отказ до проверки токена, токен цел
 - 500** сбой чтения файла
- 405 · 307 · 400 · 431** ответы самого HTTP-сервера, до NOX: чужой метод (Allow: GET, HEAD, PUT), неканонический путь, битый запрос, огромные заголовки

ОТВЕТ

```
HTTP/1.1 200 OK
Content-Type: image/jpeg
Content-Length: 184320
Accept-Ranges: bytes
Last-Modified: Sun, 27 Sep 2026 10:15:02 GMT

<184320 байт файла photo.jpg>
```

ПОЛЯ ОТВЕТА

Content-Type	mime, который заявил отправитель; не проверяется
Content-Length	размер файла или диапазона
Accept-Ranges	bytes
Last-Modified	примерно конец загрузки
Content-Range	только у 206 и 416
ETag и др. тело	не отдаются: имя, размер и тип — в attachment сырые байты файла или диапазона; ошибка — строка text/plain

ВАЖНО

- Токен гасится до отдачи: сжигает его любой ответ после проверки, включая 304, 412 и 416. Докачка — новый file.downloadBegin плюс Range: bytes=N-.
- Срок хранения и целостность проверяет только file.downloadBegin; GET отдаёт то, что лежит на диске. На отдачу — 15 минут.
- HEAD нарочно не поддержан, чтобы случайный HEAD не сжёг одноразовую ссылку: 405 без Allow. Размер и тип до скачивания — в attachment.

GET /health **основной · TLS 1.3**

Жив ли процесс — для служб ОС и туннеля.
кто пускает: не нужна: отвечает любому, кто прошёл TLS.

ЗАПРОС

```
GET /health HTTP/1.1
Host: 192.168.1.61:8080
```

ПОЛЯ ЗАПРОСА

Host обяз. обязателен в HTTP/1.1; обработчик его не читает

тело нет

КОДЫ ОТВЕТА

200 GET или HEAD (у HEAD — без тела)

405 · **307** · **400** ответы самого HTTP-сервера, до NOX: чужой метод (Allow: GET, HEAD), неканонический путь, битый запрос

ОТВЕТ

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 15

{
  "status": "ok"
}
```

ПОЛЯ ОТВЕТА

Content-Type application/json

тело {"status": "ok"} — всегда эта константа, других полей нет

ВАЖНО

- Это живость, а не готовность: база не трогается. 200 — процесс запустился и принимает TLS.
- Сертификат самоподписанный: из консоли — curl -k или проверка пина. /health/ со слэшем — 404.
- Программам — только этот ответ; служебную страницу не парсить.

GET / служебный · HTTP

Служебная страница для человека у машины: claim-ссылка с QR, пока устройств нет, потом — сводка. кто пускает: нет; слушает только loopback, а Host не localhost — 403 (защита от DNS-rebinding).

ЗАПРОС

```
GET / HTTP/1.1
Host: 127.0.0.1:8081
```

ПОЛЯ ЗАПРОСА

Host обяз. только localhost или loopback-адрес, иначе 403

тело нет

КОДЫ ОТВЕТА

200 страница отрисована; HEAD — те же заголовки без тела

403 Host не localhost и не loopback — проверяется раньше пути

404 путь не «/»

500 не удалось прочитать состояние, выпустить claim-токен или отрисовать страницу

405 · **307** · **400** ответы самого HTTP-сервера, до NOX: чужой метод (Allow: GET, HEAD), неканонический путь, битый запрос

ОТВЕТ

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Cache-Control: no-store
Content-Security-Policy: default-src 'none'; ...
Referrer-Policy: no-referrer
X-Frame-Options: DENY
X-Content-Type-Options: nosniff

<HTML-страница: QR, claim-ссылка, кнопка Сору>
```

ПОЛЯ ОТВЕТА

Cache-Control no-store — claim-ссылку не кэшировать

Content-Security-Policy по умолчанию всё запрещено; разрешены встроенные стили, картинки data: и скрипт кнопки Сору по хешу

Referrer-Policy no-referrer — ссылка не утекает

X-Frame-Options DENY — во фрейм не встроить

тело HTML, не контракт. Нет устройств — claim-ссылка, QR (если адрес доступен из сети), Сору. Есть — версия, аптайм, схема, размер базы, число устройств, чатов и сообщений; обновляется раз в 10 с

ВАЖНО

- Не контракт: разметку не парсить, программам — GET /health.
- Порт открыт только на loopback и без TLS — сознательно; если он занят, сервер работает без страницы.
- Claim-токен без срока. Пока устройств нет, сервер выпускает новый на каждом старте и показывает его; успешный claim гасит все прочие.

ТРАНСПОРТ

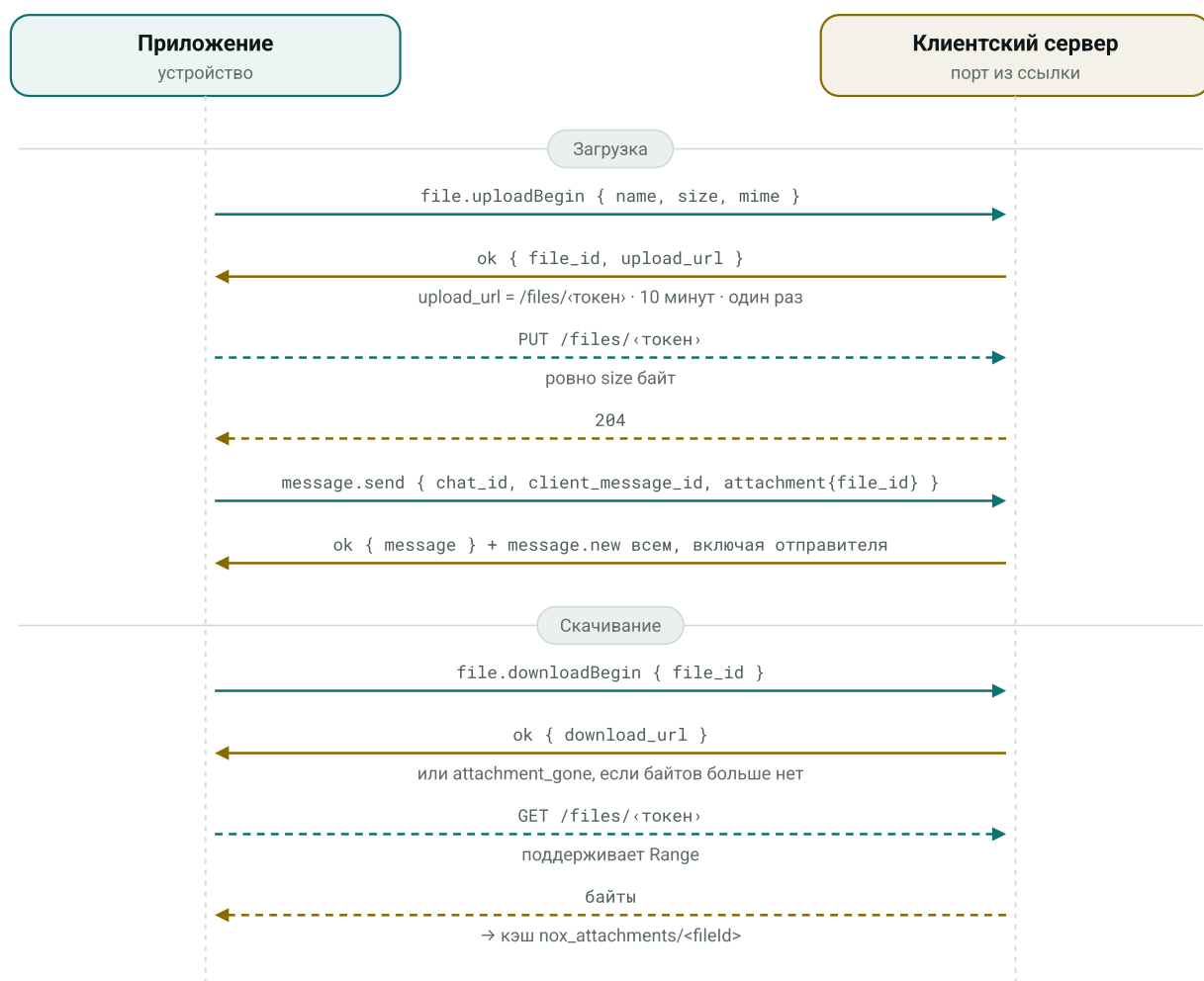
Основной порт — только TLS 1.3 и HTTP/1.1, HTTP/2 выключен. Сертификат самоподписанный, ключ ECDSA P-256; доверие — только пин sha256(SPKI) из ссылки спаривания, имя и срок не проверяются. На рукопожатие и заголовки — 5 с, на тело файла — 15 минут. Служебный порт — открытый HTTP, только на loopback.

ТОКЕНЫ ПЕРЕДАЧИ

32 случайных байта, одноразовые, живут 10 минут и только в памяти сервера — рестарт их обнуляет. Любая ошибка токена — 404 без объяснения причины, чтобы по ответу нельзя было проверить, существует ли токен.

Файлы

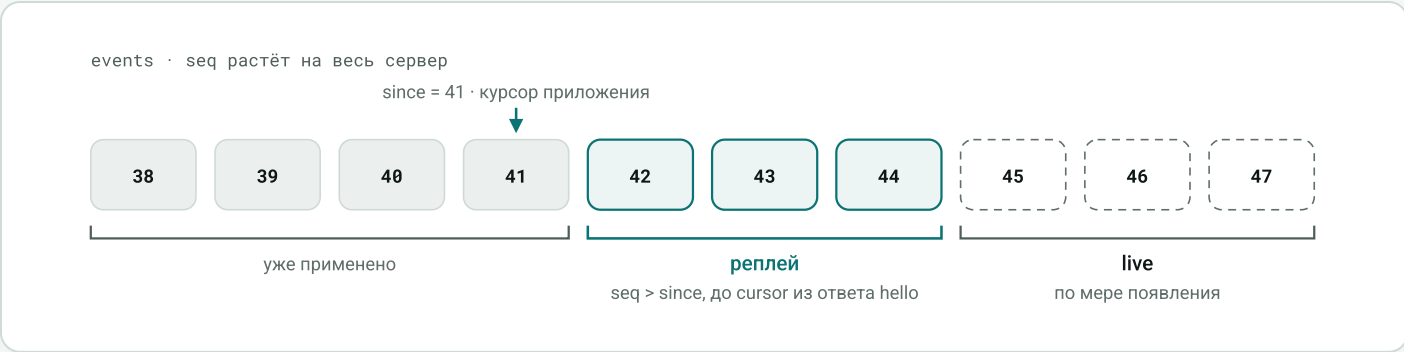
Пропуск — по WebSocket, байты — по HTTPS. Сплошная стрелка — WebSocket, пунктир — HTTPS; цвет — кто отправляет.



Приходящие картинки приложение скачивает само, остальные файлы — по нажатию. Загрузки, которые за 24 часа не привязали к сообщению, сервер удаляет при старте.

Синхронизация

Каждое изменение переписки — новый чат, переименование, сообщение — пишет событие в журнал в той же транзакции. Порядок seq — единственный порядок, который знает приложение.



Дубли на стыке реплея и live допустимы — приложение их отбрасывает по seq. Пропусков быть не может.

journal_id

Имя хранилища сервера. Если в ответе на hello оно другое, приложение **стирает** локальные чаты, сообщения и курсор и подключается заново — это другой мир.

Первое подключение

Курсор 0 — реплея нет. Приложение берёт cursor из ответа и загружает данные через `chats.list` и `messages.list`.

Медленный клиент

Если очередь соединения переполнилась (64 сообщения), сервер закрывает сокет. Приложение переподключается, и реплей закрывает дыру.

Ошибки

Код ошибки приходит в ответе: `{"ok": false, "error": {"code", "message"}}`. Незнакомый код приложение считает `internal`.

код	когда	что делает приложение
команды		
invalid_request	неверные данные, неизвестная команда, команда до hello, повторный hello	без повтора; на hello — терминальная фаза
not_found	чата или файла нет	без повтора
name_taken	имя чата занято (без учёта регистра)	подсказка у поля имени
payload_too_large	тело > 64 KiB или файл > 100 MiB	без повтора; файл отсекается заранее по limits, текст — нет
attachment_gone	байтов файла больше нет	экран файла: файл недоступен
internal	сбой на сервере — и любой неизвестный код	повтор с паузой
вход		
unsupported_schema	версия схемы в hello не 1	терминально, без переподключения
unauthenticated	нет подписи, подпись неверна, ключ неизвестен	как отзыв: выход с полной очисткой
invalid_token	токен потрачен, неизвестен или claim при живых устройствах	вход откатывается
token_expired	приглашению больше 10 минут	вход откатывается

код	когда	что делает приложение
rate_limited	есть в контракте, сервер никогда не шлёт	до трёх повторов
ТРАНСПОРТ		
HTTP 404 /files	любой сбой токена передачи — причина не раскрывается	загрузка: один повтор с новым пропуском
HTTP 413 / 400	байт больше / меньше заявленного	без повтора
пин не сошёлся	по адресу ответил чужой ключ	баннер о чужом сервере, ничего не стирается

Лимиты

Первые три сервер сообщает в ответе на hello. До отправки приложение проверяет только размер файла.

- сообщение в чате ≤ 64 KiB — весь body в JSON, не только текст
- файл ≤ 100 MiB
- WebSocket-сообщение целиком ≤ 128 KiB
- ping каждые 25 с
- ответ на команду ≤ 10 с
- переподключение 1 → 30 с ± 20%
- страница ≤ 100 · приложение берёт 20
- имя чата 1–64 символа
- токен передачи — 10 мин, один раз
- приглашение — 10 мин
- claim — без срока
- вложение хранится 10 лет
- rate limiting на сервере нет

Какие данные куда едут

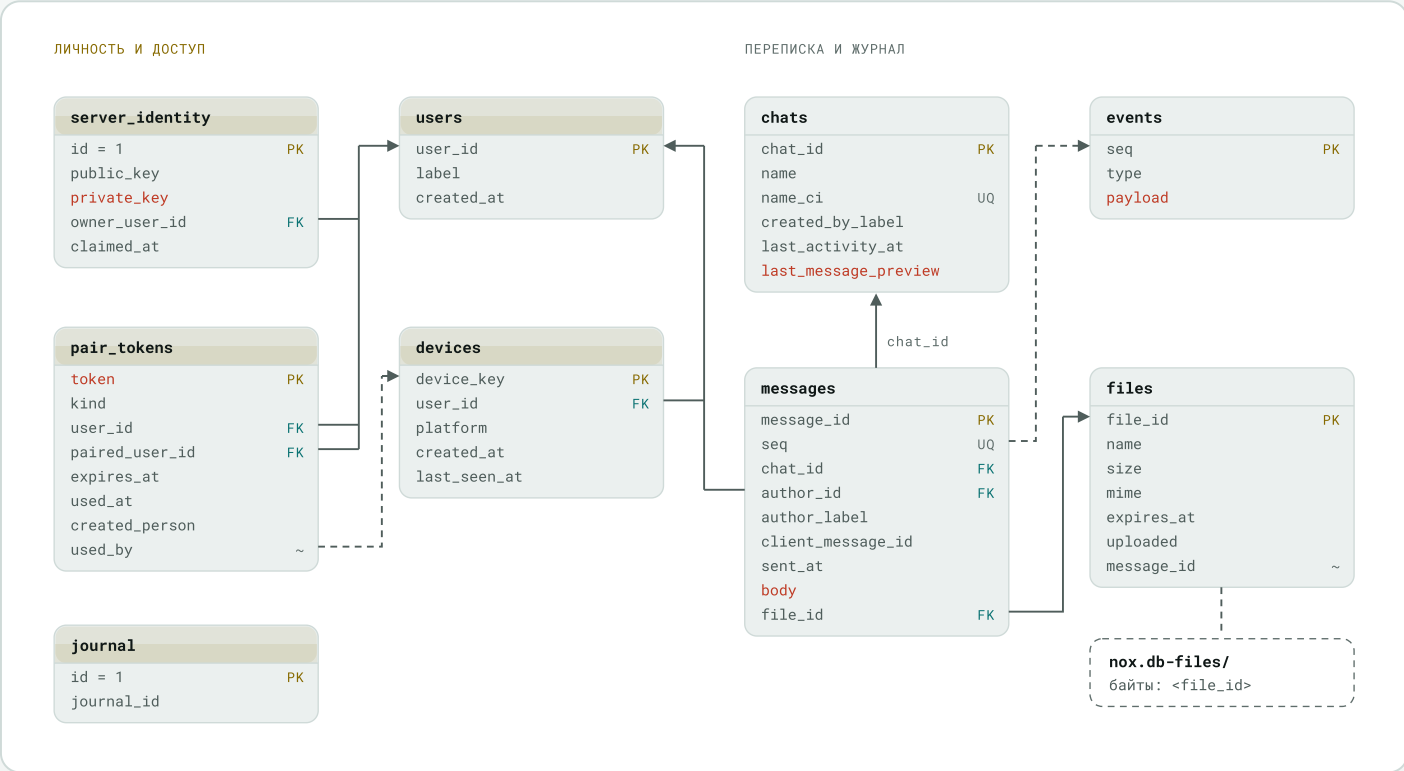
Каждая строка — один вид данных: где он лежит на устройстве, в каких сообщениях идёт по проводу и где оседает на сервере.

данные	на устройстве	по проводу	на сервере
Ключ устройства · приватный не покидает устройство	хранилище ОС · session.device_secret	— никогда	— никогда
Ключ устройства · открытый	выводится из приватного	pair · session.hello · device.revoke	devices.device_key
Подпись вызова	считается на каждом подключении	session.hello	проверяется, не хранится
Ключ сервера · приватный не покидает сервер	— никогда	— никогда	server_identity.private_key
Отпечаток сервера	хранилище ОС · session.server_fingerprint	в ссылке спаривания	считается на лету
Адрес сервера	хранилище ОС · session.server_address	в ссылке спаривания	задаётся при запуске сервера
Токен спаривания на сервере — открыто	хранилище ОС · session.identifier	ссылка · pair	pair_tokens.token
Платформа устройства	—	pair	devices.platform
Имя человека	Prefs · session.label	ответы hello и pair · identity.setLabel · identity.updated	users.label + копии в чатах и сообщениях
ID человека u_...	Prefs · session.author_id	ответы hello и pair · author_id	users.user_id

данные	на устройстве	по проводу	на сервере
Чаты	Sembast · chats	chats.list · chat.* · chat.created · chat.updated	chats
Текст сообщения на сервере — открыто	Sembast · messages; неотправленные — outbox	message.send · messages.list · message.new	messages.body, chats.last_message_preview (первые 120 символов) и events.payload
Метаданные файла	поля attachment_* в messages	file.uploadBegin · attachment в сообщении	files
Байты файла	кэш nox_attachments/	HTTPS PUT / GET /files/<токен>	nox.db-files/
Курсор синхронизации	Sembast · sync.state.since	since в hello · seq событий	events.seq
Имя хранилища journal_id	Sembast · sync.journal	ответ hello	journal
Настройки: тема, язык, уведомления не покидает устройство	Prefs	— никогда	— никогда

База сервера

Один файл SQLite (nox.db , режим WAL), одна миграция 001_init.sql . Байты
файлов лежат рядом, в каталоге nox.db-files/ .



PK — ключ · FK — ссылка · UQ — уникально · ~ — логическая ссылка без
внешнего ключа. Красным — чувствительные данные, которые хранятся
открыто. Удаляет сервер только строки devices (отзыв) и files (незавязанные
загрузки); всё остальное только дописывается или обновляется.

Все колонки всех таблиц			—
users — один человек этой машины — вторая строка физически невозможна			
user_id	TEXT	PK	u_ + 16 hex

label	TEXT	имя, не уникально
created_at	INTEGER	unix-секунды

devices — список допущенных устройств; отзыв удаляет строку

device_key	TEXT PK	открытый ключ Ed25519, base64
user_id	TEXT FK	→ users
platform	TEXT	из pair
created_at	INTEGER	
last_seen_at	INTEGER	при каждом hello

server_identity — ключ машины — одна строка

id	INTEGER PK	= 1
public_key	TEXT	SPKI DER, base64
private_key	TEXT	PKCS#8 DER, base64, не зашифрован
owner_user_id	TEXT FK?	→ users, кто спарился первым
claimed_at	INTEGER?	только отметка

pair_tokens — одноразовые токены: claim и приглашения

token	TEXT PK	16 байт, base64url, открыто
kind	TEXT	'claim' 'invite_device'
user_id	TEXT FK?	чьё приглашение
expires_at	INTEGER?	claim — NULL, приглашение +600 с
used_at	INTEGER?	потрачен
used_by	TEXT?	device_key, без FK
paired_user_id	TEXT FK?	кого дал
created_person	INTEGER	1, если создан человек
created_at	INTEGER	

journal — имя хранилища — одна строка

id	INTEGER PK	= 1
journal_id	TEXT	j_ + 16 hex

chats — чаты этого сервера

chat_id	TEXT PK	c_ + 16 hex
name	TEXT	1–64 символа
name_ci	TEXT UQ	имя без регистра
created_at	INTEGER	
created_by_label	TEXT	копия имени автора
last_activity_at	INTEGER	растёт при отправке
last_message_preview	TEXT	≤ 120 символов

messages — история

message_id	TEXT PK	m_ + 16 hex
seq	INTEGER UQ	= seq своего события
chat_id	TEXT FK	→ chats
author_id	TEXT FK	→ users
author_label	TEXT	копия имени на момент отправки
client_message_id	TEXT	с author_id — ключ идемпотентности
sent_at	INTEGER	
body	TEXT	JSON клиента ≤ 64 KiB, открытым текстом
file_id	TEXT FK?	→ files, один файл — одно сообщение

files — метаданные вложений; байты — в nox.db-files/

file_id	TEXT PK	f_ + 16 hex
name	TEXT	≤ 255
size	INTEGER	≤ 100 MiB
mime	TEXT	≤ 128
created_at	INTEGER	
expires_at	INTEGER	+10 лет
uploaded	INTEGER	1 после записи байтов
message_id	TEXT?	обратная ссылка, без FK

events — журнал: полный порядок для реплея

seq	INTEGER PK	AUTOINCREMENT, растёт на весь сервер
type	TEXT	chat.created chat.updated message.new
payload	TEXT	JSON-снимок на момент записи

Хранилища приложения

Три места — по чувствительности данных.

■ Хранилище ОС

`flutter_secure_storage` — всё, что даёт право войти

`session.device_secret`

приватный ключ устройства, seed Ed25519. Не покидает устройство

`session.identifier`

потраченный токен из ссылки; его наличие и есть «сессия есть»

`session.server_address`

host:port из ссылки

`session.server_fingerprint`

пин: base64 от sha256 ключа сервера

`auth_id_token`

зарезервирован, никто не пишет

Выход стирает все ключи сессии. Новый вход — новый ключ устройства.

■ SharedPreferences

открытые данные

`session.label`

имя человека

`session.author_id`

id человека u_..., отличать свои

сообщения

`session.onboarding_complete`

имя выбрано

`settings.theme_mode`

тема

`settings.notifications_enabled`

уведомления

`ui_language`

язык интерфейса

Первые три стираются при выходе, настройки остаются.

■ Sembast · app_dev.db

кэш данных сервера (на prod-флейворе — app.db с моками)

`chats`

чаты и отметка прочтения

`last_opened_seq`

`messages`

сообщения всех чатов и путь к скачанному файлу

`outbox`

очередь отправки; ключ записи =

`client_message_id`

`sync`

`state{since}` — `cursor · epoch · journal{id}`

Плюс кэш файлов

`<cache>/nox_attachments/<fileId>.<расширени`

Выход очищает всё, кроме файла базы и записей epoch и journal в sync — они нарочно переживают выход.